EPFL - Computer Science

COM-402 Information Security and Privacy

Lecture 03 — September 29, 2025

Author: Jonathan Dutoit

Professor: Mathis Payer

Access Control

Learning Goals

- Know authentication factors (password, device, biometrics), strengths and risks
- Differentiate and identify access control policies (RBAC, DAC, MAC)
- Understand the value of authentication protocols and delegated authentication

Authentication

What is authentication?

Authentication is the process of **verifying** someone's or something's **identity**

Identification is the act of identifying a particular user (a subject), often named through a username.

Password-Based Authentication

- Simplest form of *Authentication*
- User sends both username and password to the server, server authenticates user's identity.

What if the network is compromised? What if the server is compromised?

Authentication Factors

Access control only makes sense if **subjects** are authenticated.

There are 3 common flavours to authenticate subjects

- Something you **know**: password, pin codes
- Something you **own**: paper card, smartphone, certificate, electronic token

- Something you **are**: biometrics

Two factor authentication (aka 2FA) requires the use of two factors

- A password is often one of the two factors

The Password is Dead...

"They just don't meet the challenge for anything you really want to secure" (Bill Gates, 2004)

"Within 5 years, you'll never need a password again" (IBM, 2011)

"Passwords are done at Google" (H. Adjins, manager of Information Security at Google, 2013)

Truth: passwords are still the main form of authentication on the web.

"No other single technology matches their combination of cost, immediacy and convenience" (C. Herley, P. van Oorschot)

Even Strong Passwords Are at Risk

A stolen password can be replayed by anybody

An attacker can:

- Steal passwords using client-side malware
- Obtain passwords by cracking hashes stolen from a server
- Phish passwords with a fake website
- Eavesdrop the password (remember, that's why we need TLS)

Attack vector: "Credential stuffing"

- Lists of usernames and passwords are distributed online
- Hackers try the same credentials on many different sites
- Sony and Gawker were both hacked: "Two thirds of people with accounts at both Sony and Gawker reused their passwords"

Password Managers

Store all your credentials in encrypted form

- Your **master password** is used to decrypt the credentials
- To be used on different devices, the encrypted credentials must be accessible online

Where are passwords stored?

- Password manager works **offline**, uses a local file (e.g., Keepass)
  - You can choose to hist this file in the cloud
- Password manager talks to **a server in the cloud** (e.g., Lastpass)
  - Browser plug-in or app downloads credentials and decrypts them locally

  - You must trust them to not steal your master password
- Best of both: the password manager comes with an open source server that you can host where you want (e.g., Bitwarden)

Authentication/Session Cookies

After logging in a web application the server sends a cookie, stored in the client

The cookie is kept by the client and used for authenticating against the server

The cookie should be unique for every subject

The cookie can be used by illegal clients through forging

- If the server encrypt the cookie it limits forging (HMAC)

Something You Own

Typically called a (hardware/software) token

Bingo card

- Proves that the user owns the card
- Can be easily copied without being detected

One time password (OTP) token

- Displays a password to be used only once
- The password changes with time or click
- Proves that user owns token at time of login
- Cannot be copied easily (secure hardware)
- OATH standard

TAN generator

- A calculator that generates a number based on user input
- Can be used to sign a transaction (Transaction Authentication Number, TAN)
- Proves that user owns the generator
- Based on a smartcard (secure hardware), typically your bank card

Smartphone

- OTP is sent by SMS or
- OTP is generated by app, as with an OTP token
  - Attack: clone the seed and then attack the 2nd factor that way. If the cost is higher than the benefit of security, then users are going to find a way around it (often less safer)
- Proof that user owns the phone (or sim card, secure hardware?)

What are the tradeoffs?

A private key in a hardware token

- Token signs a challenge
- Different keys for different websites
- Proof that user owns the token
- Secure hardware
- Universal 2nd Factors (U2F) standard

OATH, Generation of OTP

OATH (initiative for open authentication) is a standard that describes:

- how OTPs are generated from a seed
- an XML format for importing the seeds into an authentication server

Standard OATH tokens exist in both hardware or software form

- E.g., the Google Authenticator app

Generation of the next OTP is either counter or time based (RFC 4226)

OATH Algorithms

Counter based:

$$hotp(k, c) = truncate(hmac - sha512(k, c)).$$

Time-based (initial time $t0$, time interval x):

$$totp(k, t) = hotp(k, (t - t0)/x).$$

What are the tradeoffs between hotp and top?

Discussion:

Counter based: it needs to be synchronized. They could be out of sync due to individual operations that could lead out of sync scenarios

Time-based: suppose the time is synchronized, replay attacks become a problem if the time interval is big enough.

RSA SecureIDs

Secure IDs implement a form of totp

Requires a random seed to be synced between devices and customers

In a massive hack, attackers stole ALL seeds from RSA. The first true supply chain attack

U2F, FIDO2

Universal 2nd Factor is a standard developed by the **FIDO** alliance

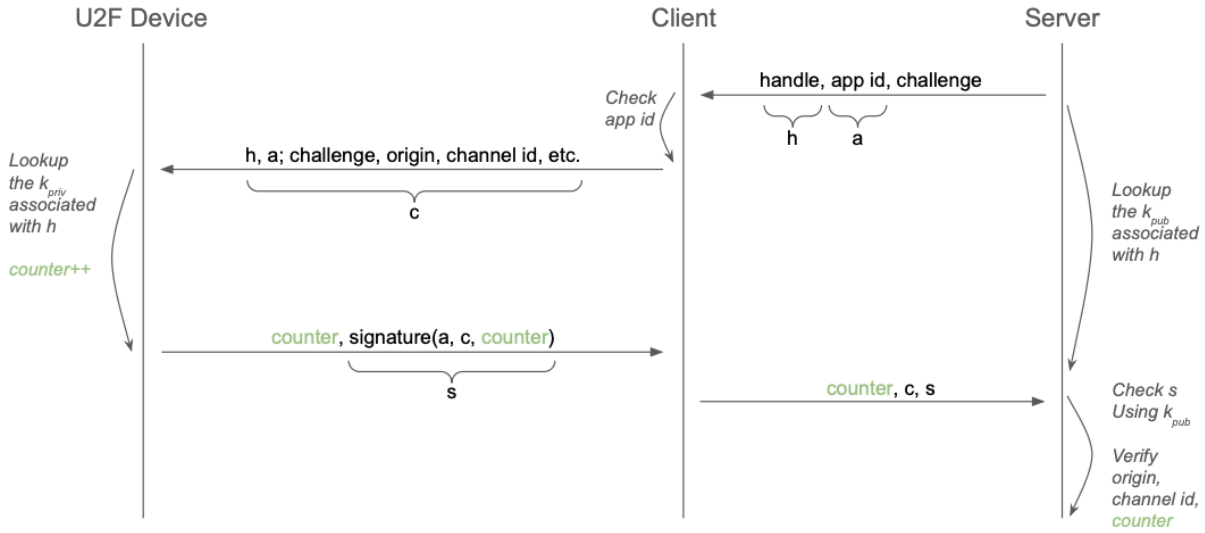
Latest version is FIDO2

For each application, the token generates a key pair and gives the public key to the server

- On login, the server sends a random challenge to the client
- The client signs (the challenge + the domain name of the server + a signature counter)
- The client sends the data and signature to the server
- The server verifies the signature

Adding the domain prevents phising attacks

Adding a counter detects cloning of the private key/replay attack



U2F, FIDo2 (WebAuthn)

FIDO2 standardizes U2F within the browser with JavaScript (called WebAuthn)

Additionally makes use of **CTAP** to access the signature

CTAP, the Client to Authenticator Protocol, describes how an application can ask an authenticator to generate an assertion (signature)

- Authenticator can be a
  - USB token
  - Smartphone connected by Bluetooth
  - Authentication module of platform (e.g., phone's biometric authentication)

The assertion contains information whether the user

- was present (e.g., clicked on a button)
- was verified (e.g., pin or fingerprint)

U2F, FIDO2 Pros

No problem if the server gets hacked:

- It is an asymmetric system. The information stored on the server cannot be used to log in

No problem if the client gets hacked

- The private key stays in secure hardware of the client
- Usage of the key is only possible with user interaction

Very convenient

- It can use the native authentication system of the platform (iPhone faceID, Microsfot Hello biometrics, Google fingerprint)

Biometrics

Motivation

- Something you know could be guessed
- Something you own could be stolen
- Nothing to remember, nothing that can be lost

Physiological

- Iris
- Retina
- Fingerprint
- Shape of head
- Shape of hand

Behavioral

- Speech
- Keystroke timing
- Handwritten signature on tablet
- Gait (way of walking)

Biometrics: Authentication Process

Registration

- Acquisition

- Extraction of characteristics
- Storage of characteristics

Authentication

- Acquisition
- Comparison
- Decision

Limitations

- Acquisition is never exact
- Comparison is never a perfect match
  - Biometric information cannot be hashed (due to imprecision in reading out data)
- Decision is always error prone

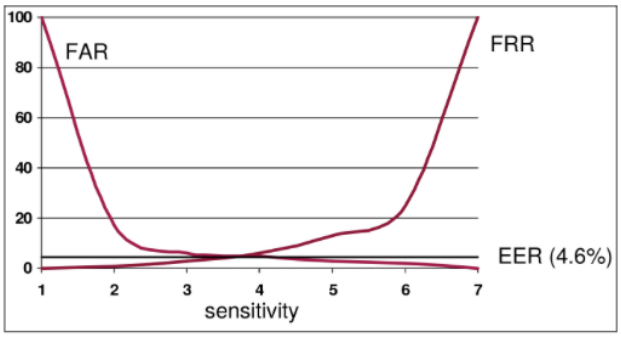
Biometrics: Error Rates

The decision algorithm must accept a certain error; sensitivity can be tuned

FAR False acceptance rate: the system declares a match when it wasn't

FFR False rejection rate: the system declares a non-match although it was a match

EER Equal error rate, when the system is tuned such that FAR=FRR



Biometrics: Fingerprints

The fingerprint is read by a sensor

An image of the ridges is created

The minutiae are extracted at

- termination of a ridge
- bifurcation of a ridge

A list of coordinates (x, y) and angles is generated

The list is compared with a stored list

- the minutiae are shifted and rotated to get the best match

Biometrics: FaceID

30,000 points are projected on the face and read from a different angle

A specialized and isolated processor constructs a 3D images and compares it to the registered face. Another one is used to store and compare images

- The main processor never sees the 3D data

The communication with the sensor (camera) is encrypted

- cannot be intercepted and replayed

This is the same for iPhone fingerprint scanning (Touch ID)

- this is why you can't just replace a broken sensor
- it has to be paired with the processor

Note: you still need to type a password when booting the phone

- used to decrypt the storage

Limitations: makeup, covid, etc...

Biometrics Discussion

Biometrics and authentication

- No hashing possible, risk: theft
- It is impossible to change a stolen fingerprint!
- Best to store the biometric data locally in protected hardware
- Example:
  - Biometrics are stored in passports, not at the customs offices
  - Smartphones store fingerprint data in separate, secure hardware
- Some sensors can be fooled or replaced
- Not ideal for remote authentication
  - Rather use it for local access to authentication key (e.g., U2F)

Biometrics and privacy

- Biometric data is considered sensitive data by European data protection laws and the Swiss law
- Biometric data can reveal health issues (e.g., eye pathologies)
- Biometrics data can exclude some people (absence of fingerprints or fingers)

- A lot of more or less serious research is done with ML and AI to extract information from faces
  - Detection of Propensity for Agression based on Facial Structure
  - Privacy fears as Tokyo taxis use facil recognition cameras to guess riders' age and gender for targeted advertisements

Access control

Access Control Basics

After we **authenticate** a subject...

Access control defines and enforces **operations** that **subjects** can do on **objects**

- Bob (subject) has permission to read/write (operation) from a socket (object)
- Implies that the subject has been authenticated first

Security Policy

Access rights (aka permissions, privileges) describe which subjects can do what operations on what objects

A **security policy** is a collection of access rights. Security policies can be represented as an **access control matrix**

Security Mechanism

Security mechanisms try to prevent operations that are not authorized by the security policy

For example, the kernel implements an access check to see if a user has permissions to open a file. If the access check fails, the rights are not granted.

Security check is only made when you open the file, the rest of the operations on the file are not checked.

Principle of Least Privilege (PoLP)

- Subjects only have the minimum rights required for their job
- I.e., subjects are only allowed minimal operations on objects per task
- This limits the impact if anything should go wrong

The challenge in access control is to have a system that is simple to implement and manage and that is close to the principle of least privilege

There is no "one size fits all" solution and often different approaches to access control are combined to achieve the best results

Most important access control principle

Multiple Levels of Access Control

Network level access control

- Subjects are connections or data packets
- They are identified by source/destination IP addresses and protocol ports
- Typical operations are pass, block, or tag
- Example: a database server only accepts traffic from inside EPFL (source addr 128.*.*.*) connecting to TCP port 3306 (mysql)

Typically enforced with:

- Network equipment: firewalls
- The servers:
  - Local firewall on the server (e.g., ufw in Linux)
  - Configuration of the server sofware

What are the tradeoffs of filtering at the different levels

Access control at the operating system

- Which user can star/stop the DB engine?
- Who can read/modify the files of the DB?

Access control in the application

- Which user of the application can edit user profiles?
- Who can see financial data?

Access control within the enterprise

- Which employees can access the application?
- Which application are limited to human resources, which to marketing?

Multiple Approaches to Organize Access Control

Three common varieties:

- Role-based Access Control (RBAC)
- Discretionary Access Control (DAC)
- Mandatory Access Control (MAC)

Role-based Access Control (RBAC)

Simplifies the specification of permissions by grouping users into roles

Centered on user roles (a role can contain multiple permissions)

2 Moodle Course Roles

Access to individual Courses	View Courseware	Participate in activities	View Personal records	View student records	Add and edit Courseware	Edit and Course Setting	Set Course Roles
Course Administrator	✓	✓	✓	✓	✓	✓	✓
Teacher	✓	✓	✓	✓	✓		
Non-editing Teacher	✓	✓	✓	✓			
Student	✓	✓	✓				
Guest Read only access	✓						

Example: Operating Systems

- Most operating systems have the notion of groups
- Groups can be given a set of permissions
- Users can be added to groups
- Examples:
  - Debian/Ubuntu: audio group can access mic and loudspeakers, wireshark group can sniff network traffic
  - Windows: ”remote desktop user” group can access desktop remotely

Which groups does your user belong to?

```
id <username>
```

Pros:

- Easy to grasp the idea of roles
- Easy to manage
  - Roles decouple digital entities from permissions
  - Simply assign roles to a new subject. No need to decide for each object
  - Easy to revoke authorizations by removing role
- Easy to tell through roles which permissions a subject has and why.
  - Typically centrally managed

Cons

- Difficult to decide on the granularity of roles
  - Do we need different roles modifying and deleting client information?
  - Leads either to role explosion or roles that are too broad (not least privilege)
- Role meaning is fuzzy
  - Employee position in company may be different from RBAC role (think developers in same team working on different subjects)
- Unclear if roles can be shared across different departments
  - For ”manager” roles, in general, is a finance IT manager the same as marketing IT manager?

Discretionary Access Control (DAC)

Access control is at the discretion of the object owner

- Owner specifies policies to access resources it owns

Access control matrix represents rules

- stored by column: Access Control List (ACL) stored with object
- stored by row: Capabilities stored with subjects

DAC in Unix file system

Stored **in the target objects**, e,g,m in the metadata of files in the file system

Subjects are grouped in three categories: owner, group, other.

Three access rights: (r)ead, (w)rite, e(x)ecute. The three rights and three groups are stored in 9 bits

- represented as three octal digits
- owner rwx, group rx, others r: `rwx`r-x—r—— = 754
- the permissions of the first matching category dominate

ACLs in Unix: setuid/setgid

If a program has setuid bit set, it will be run with the permissions of the owner of the file instead of the permissions of the user running the program

Very useful to give users more privileges in specific cases

Example: the passwd command allows a user to modify the `/etc/passwd` and `/etc/shadow` files

```
ls -l /etc/{passwd,shadow}
-rw-r--r-- 1 root root 2786 avr 23 11:09 /etc/passwd
-rw-r----- 1 root shadow 1489 avr 23 11:09 /etc/shadow
```

and the program `/usr/bin/passwd` has the setuid bit

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 59640 jan 25 2018 /usr/bin/passwd
```

Hence when a user Jane runs the program passwd, the process is run as root.

The setgid bit does the same for groups: the group of the process running the program is set to the group of the owner of the program.

setuid and setgid are displayed as s instead of x in the access rights of the file.

setuid is very practical, because it lets unprivileged users execute some well defined privileged actions, but it can also be dangerous, why?

⇒ Any bug in programs where the setuid bit is set could be used to escalate the privileges of the user. As the user controls inputs to the program running as root, they may use that input to trigger the bug and gain arbitrary code execution.

Capabilities in Linux

Capabilities are permissions that are related to a subject, not to an object

Linux supports capabilities for processes. Some example are:

- `CAP_CHOWN`: make arbitrary changes to file user ID and group ID
- `CAP_SYS_BOOT`: user reboot or kexec (load a new kernel)

Example: dumpcap is the program used by wireshark to sniff network traffic

- it can only be run by user root and member of the wireshark group
- It does not have the setuid bit

```
$ ls -l /usr/bin/dumpcap
-rwxr-xr-- 1 root wireshark 104688 jan 19 06:23 /usr/bin/dumpcap
```

Let's check the capabilities of dumpcap:

```
$ sudo getcap /usr/bin/dumpcap
/usr/bin/dumpcap = cap_net_admin,cap_net_raw+eip
```

The process assumes `net_admin` and `net_raw` capabilities when launching `dumpcap`: it can read and write to all network interfaces

The program can do this while running in the name of the user:

```
$ ps -ef | grep dumpcap
user 24342 26651 0 13:10 pts/1 00:00:00 /usr/bin/dumpcap
```

This is much safer than using setuid

If there was a bug in dumpcap allowing to execute arbitrary commands, setuid would run these commands as root!

DAC Pros and Cons

Pros

- Flexible
- Intuitive
- Easy to manage (owners get to set the permissions themselves)

Cons

- Depends on the owners judgement
- Only works if programs are benign and users make no mistakes
- Vulnerable to the "Trojan" / declassification problem

Mandatory Access Control

Tries to ensure that even someone with access cannot leak the data

Historically associated with military-grade information security

- Multilevel security: e.g., unclassified, confidential, secret, top-secret

The system labels both subjects and objects with security labels

- Can only be modified by trusted administrators via trusted software

Security policy:

- Example: Subjects can only access objects of the same or lower level

Depends on trusted software and admins for

- Keeping the system in a protected state, by preventing operations that violate the rules of the matrix
- Labeling new subjects and objects
- Perform transitions of labels (e.g., when a document is declassified)

CAN be used in conjunction with DAC or RBAC

Quiz! Why is it a bad idea to allow secret subjects to write confidential documents?

Remember the order of "secrecy": unclassified, confidential, secret, top-secret

A secret subject may read a secret file and write the content into a confidential file, thereby reclassifying the secrecy level and downgrading it. This is also called the "no-write-down" problem.

MAC Confidentiality vs Integrity

MAC and Confidentiality

- When protecting confidentiality, we don't want users to write to a lower level (**no write-down)**
  - Prevents leaking information from higher to lower levels ("trojan")
- Typical scenario: network access control
  - Network split in zones: internet, internal, secret
  - Firewalls only allow data to flow from lower zones to higher zones

MAC and Integrity

- We don't want users from lower levels to write into higher levels (**no write-up**)
  - Prevents unauthorized modification of objects
- Typical scenarios: operating systems
  - Users can read and execute OS programs, but they cannot modify them

Pros

- Addresses the limitations of DAC
- Easy to scale

Cons

- Can be too restrictive, prevent legitimate tasks
- Not flexible

Authentication protocols

When sending a password over TLS is not enough

Motivation

Why is the naive Password-based Authentication insufficient?

- Sending passwords over network is risky, leaks to adversary when the network is compromised
- Attacker may break encryption
- Server may be compromised or even store passwords twoo
- Challenge-response Authentication

Why do we need authentication protocols?

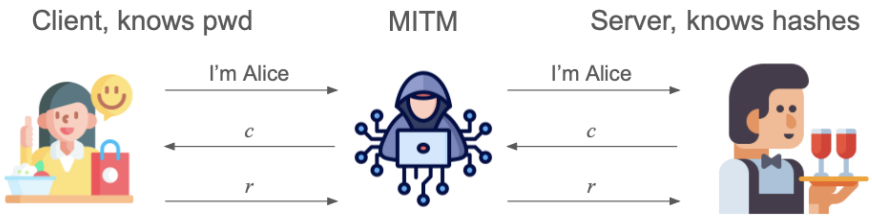
- Standardization and save duplicated efforts implementing authentication
- Kerberos

Challenge-Response

Rather than sending the password to the server

- The server sends a random challenge to the client
- The client uses the password hash to create a response
  - E.g., encryption or HMAC of the challenge

Typically, challenge response protocols are vulnerable to MITM attacks



Microsoft introduced the signature of the packets with a key derived from the pwd hash (it is actually an HMAC)

- The MITM does not know the key, cannot send any packets

Mutual Authentication

The server and client can both use a challenge: they can authenticate each other.

Examples:

- Newer versions of Microsoft challenge-response (SMBv2)
- WiFi WPA, WPA2, and WPA3

Challenge-response protocols may be eavesdropped or suffer cracking attacks

- The attacker records a challenge and a response
- They try all possible password to find which would yield the same response

Kerberos

Kerberos provides authentication and authorization across a network

Subjects receive tickets that they use to access objects

Exclusively based on symmetric keys

Developed at MIT in the 80's, initially deployed in Unix, the main authentication protocol in Windows LAN networks.

Key idea: delegation

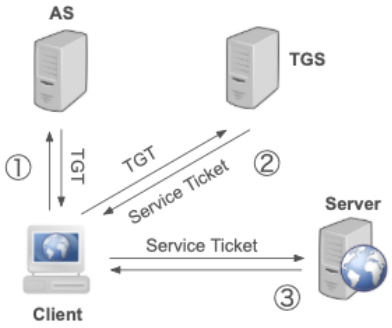
- Users authenticate once and then access **multiple services** without needing to repeatedly re-enter credentials

Separation of concerns

- Authentication
- Access control (Authorization)
- Providing actual service

Kerberos uses a three-phase approach

①: an authentication server (AS) authenticates the client and delivers a ticket granting ticket (TGT)
②: the client can then present the TGT to the ticket granting server (TGS) to get a service-specific ticket
③: the client can access the service



The **AS** handles **authentication** (user identity verification and issuing the TGT).

- This step is solely about **confirming** the user's **identity**.

The **TGS** handles **access control** (authorization, issuing service-specific tickets based on the TGT)

- Once the TGT is issued by the AS, the user doesn't need to go back to the AS for each service access. Instead, the TGT can be used multiple times to request service-specific tickets from the TGS.

Delegated authentication

OAUTH2

Kerberos is great but targets the intranet. Following the idea of delegated auth...

Oauth2 (RFC 6749) is used for delegated authentication on the **Internet**

- Oauth2 providers like Facebook, Google, or X/Twitter can be used to authenticate and access other applications

Remeber TLSv1.2 vs 1.3? OAUTH2 is much simpler than Kerberos

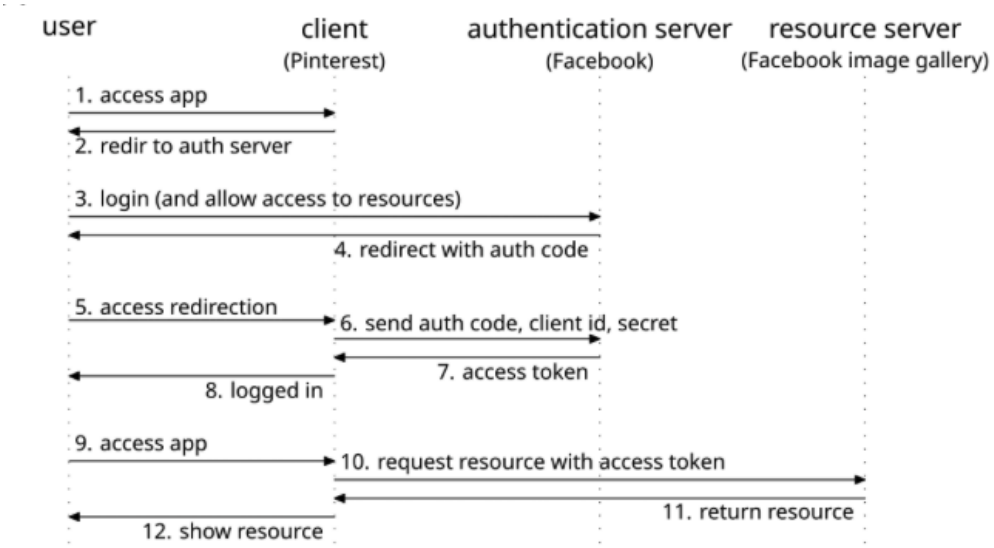
Roles

Client: application that wants to use authentication and possibly access the user's data

Resource server: server that has user's data that client wants to use

Authorization server: server on which the user authenticates

User: Owner of the account and resources on resource server



The client and the authentication server have a shared secret

- the client thus has to register with the authentication server before being able to offer this service
- the secret is used when the client exchanges the **authentication code** for an **access token**
- the authentication code is not sufficient to get access to the resources
- it can only be used by the client and nobody else

Oauth2 can be used by browsers or in app

- in an app a redirection for authentication can be either
  - opening a browser within the app (called a webview)
    - not very safe as the app could be spying while you login
  - switching to the other app (e.g. Facebook) and then back

Most apps in smartphones (e.g., X, Instagram, Gmail) do not store your password

- The use OAuth2 to request an access token and use it
- When you change your password you don't need to type your new password into all your devices